QR Code Safety Scanner & Generator

A College Project Report

Submitted By:
Candidate Name: [Your Name]
Department: Computer Science & Engineering
Roll Number: [Your Roll Number]

Submitted To:
Institution: [Your College Name]
Supervisor: [Your Guide Name]
Designation: Assistant Professor (CSE)

Abstract

With the rapid adoption of QR codes for contactless payments, menus, logins, and network sharing, they have become a prominent vector for social engineering, phishing, and malware distribution (commonly known as 'QRLJacking' or 'Quishing'). Traditional QR code scanners decode and open links blindly, exposing users to severe security vulnerabilities.

This project presents a Local-First, Client-Side QR Code Safety Scanner & Designer Generator. The system not only decodes QR payloads but subjects them to an 8-rule Heuristics Engine to calculate a real-time risk index, executes visual sandbox rendering via isolated screenshot generation, and features a WPA/WPA2/WPA3 Wi-Fi configuration auditor. Built strictly with standard web technologies (HTML5, Vanilla CSS3, ES6, and browser-native APIs), the application preserves user privacy by performing all cryptographic and safety scoring calculations locally.

Table of Contents

1. Introduction & Problem Statement ..... Page 3
2. System Design & Objectives ..... Page 3
3. System Architecture & Workflow ..... Page 4
4. Heuristic Safety Analysis Engine ..... Page 5
5. Detailed Technical Specifications ..... Page 6
6. Wi-Fi Diagnostics & Security Model ..... Page 6
7. Conclusion & Future Scope ..... Page 7

Chapter 1: Introduction & Problem Statement

A Quick Response (QR) code is a matrix barcode that stores binary data in a two-dimensional grid. Because humans cannot read these grids visually, attackers exploit this blind spot by placing malicious QR codes over legitimate ones in public places (e.g., parking meters, public transit, flyers).

The Core Threat Vectors Checked:
- Phishing Portals: Directing users to lookalike login forms to harvest credentials.
- Hidden Redirects: Using URL shorteners (e.g., bit.ly) or redirects to hide the actual landing page.
- Insecure Wi-Fi Access: Auto-connecting devices to insecure rogue network setups.
- Homograph Attacks: Registering domains withUnicode characters to spoof target brand names.

Chapter 2: System Design & Objectives

To mitigate these hazards, the application is engineered around four main system objectives:

- Zero Cloud Dependency: Perform all calculations locally in the browser sandbox to ensure privacy.
- Proactive Warning Systems: Alert users of dangerous destinations using security verdict overlays.
- High-Clarity Visualization: Integrate clear threat dials and structured logs to list vulnerabilities.
- Cryptographic QR Design: Allow users to generate highly customized QR codes, including WPA2 Wi-Fi setup and AES-256 encrypted payloads.

Chapter 3: System Architecture & Workflow

The application architecture consists of a scanner pipeline, generator pipeline, and the heuristics analyzer component. The operational flow is structured as follows:

1. Capture & Decryption: The scanner processes input from camera feeds or file selections, decrypts it using CryptoJS (AES-256) if password-protected, and extracts the payload string.

2. Analysis Routing: The system determines if the payload is a URL link, a Wi-Fi configuration string, or raw text data. Wi-Fi configurations are sent directly to the local parameters auditor, while links are passed to the unshortening and safety checkers.

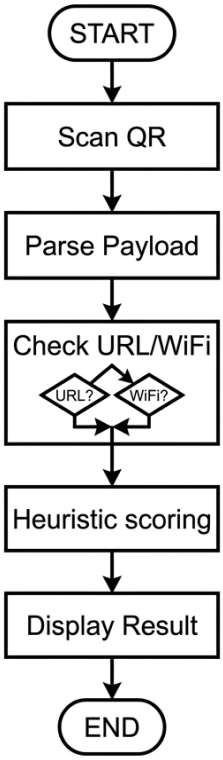


Figure 3.1: QR Safety Scanner Operational Workflow Diagram

Chapter 4: Heuristic Safety Analysis Engine

The heuristics core checks URLs against 8 checkpoints to calculate safety levels:

1. Connection Encryption: Flags insecure http: protocols (+25 risk points).
2. Host Type Check: Flags numeric raw IP address hosts (+35 risk).
3. Link Masking: Identifies URL shorteners (like bit.ly) masking domains (+20 risk).
4. Domain TLD Security: Flags low-reputation top-level domains like .xyz or .top (+25 risk).
5. Phishing Keywords: Searches paths for credential harvesting terms (e.g. login, signin) (+20 risk).
6. Subdomain Count: Flags excessive subdomain nesting (dots in hostname > 3) (+15 risk).
7. Domain Length: Identifies character length abuses (>40 characters) (+10 risk).
8. Unicode Homographs: Flags character mixing used to spoof target brand domains (+35 risk).

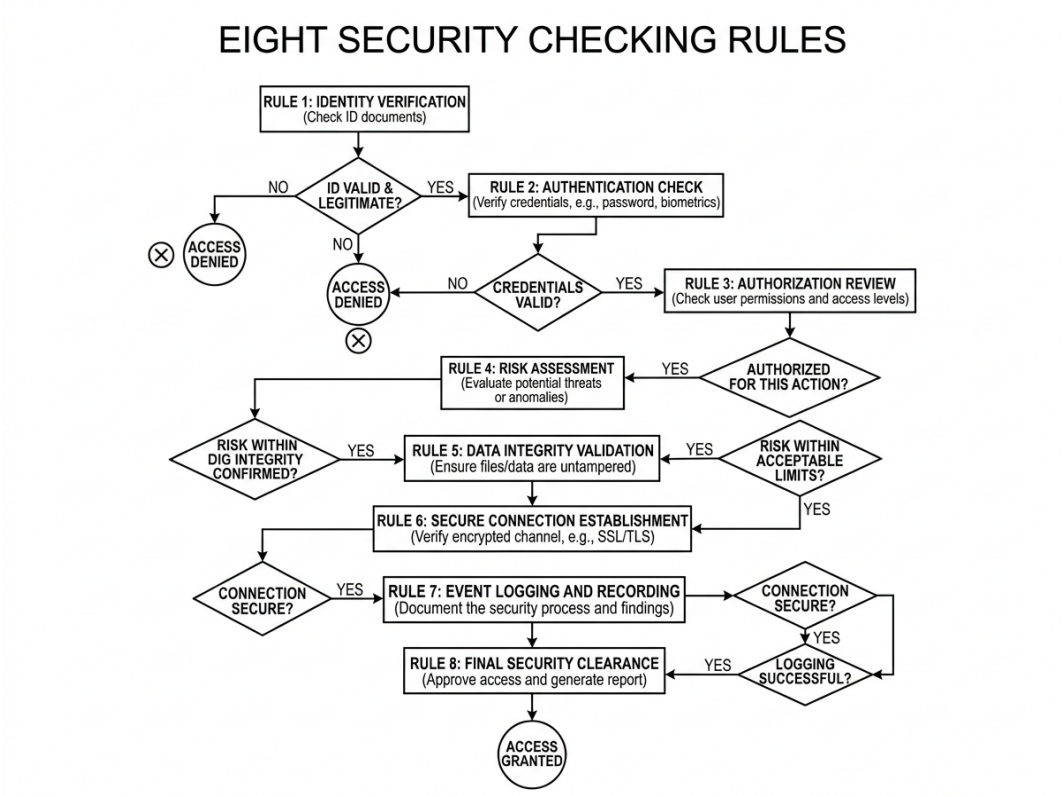


Figure 4.1: Heuristics Safety Core Engine 8-Checkpoints Flowchart

Chapter 5: Detailed Technical Specifications

The application is built strictly using modern web standards without complex compilers:
- Frontend Stack: HTML5 semantic tags for layout, CSS3 variables for theme styles, and ES6 Javascript.
- Camera Integrations: html5-qrcode library for local WebRTC media streams.
- File Matrix Decode: jsQR library for canvas byte scanning.
- Encryption / Decryption: CryptoJS for client-side AES-256 ciphers.
- Dynamic Vector Graphics: qr-code-styling for custom vector drawing.
- Document Compiling: jsPDF for client-side document compilation.

Chapter 6: Wi-Fi Diagnostics & Security Model

When scanning a Wi-Fi setup string, the scanner auditor executes the following tasks:

- Encryption Check: Verifies if connection security is Open (no security), WEP (vulnerable), or WPA/WPA2/WPA3.
- SSID Broadcast Auditor: Flags hidden networks and prints guides on how to configure manually.
- Credential Clipboard Action: Allows copy-paste of passwords and network names, minimizing typing errors.

Chapter 7: Conclusion & Future Scope

The Local-First QR Code Safety Scanner & Generator successfully demonstrates that heuristic scanning and security audits can run efficiently client-side without compromising user privacy.

Future Enhancements:
1. Offline Threat Databases: Storing light, compressed hash lists of malicious domains in IndexedDB.
2. Cryptographic QR Verification: Supporting Ed25519 digital signatures to verify the issuer authority of public QR codes.